

الفصل السادس: التعريب وتعدد اللغات، الملفات والتخزين، وملاحظات تقنية مهمة

ما ستتعلمه في هذا الفصل

- كيف يحدّد الـ backend لغة كل طلب عبر ترويسة `Accept-Language`، وما الأثر الجانبي الخفي لذلك على جدول المستخدمين.
- الفرق بين قواميس الترجمة الثلاثة الموجودة في المشروع: `lang/en.json` و `lang/ar/*.php` و الأعمدة المترجمة في قاعدة البيانات.
- كيف تعمل حزمة `spatie/laravel-translatable` هنا، وأي الموديلات والأعمدة تستخدمها.
- لماذا لا يمكن الترتيب (`sort_column`) على الأعمدة المترجمة، وما الرمز الذي يعيده الـ API في هذه الحالة.
- منظومة رفع الملفات والتخزين (`filesystems`)، وأربع دوال مساعدة غير متسقة يجب الحذر منها.
- الحقيقة الصادمة حول تحديد معدل الطلبات (rate limiting) في هذا المشروع: لا وجود له فعليًا.
- إعداد `Telescope`، ومساژه المُبهم، والفجّ الذي جعل زوّار الشهادات يحصلون على 403.
- ملفات الاختبارات الموجودة وما يغطّيه كل منها.
- الأخطاء الشائعة: متغيّر `PLATFORM` الفارغ، رموز الأخطاء غير المتوقعة (400 بدل 403)، وتسجيل الطلبات في قاعدة البيانات.

1. التعريب وتعدد اللغات (Localization)

1.1 المفهوم أولًا

في أي واجهة برمجية (API) تخدم أكثر من لغة، يجب أن يكون تحديد اللغة جزءًا من الطلب لا من حالة الخادم. الأسلوب المعياري هو أن يرسل العميل ترويسة `Accept-Language` مع كل طلب، ويقوم الخادم بضبط لغته الحالية (locale) في بداية دورة الطلب، ثم تعود جميع الرسائل والبيانات المترجمة بتلك اللغة.

في هذا المشروع يوجد ثلاث طبقات من الترجمة:

الطبقة	المكان	ما تُترجمه
رسائل النظام القصيرة	<code>lang/en.json</code> ، <code>lang/ar.json</code>	رسائل النجاح والخطأ التي يعيدها الـ API
قواميس PHP لكل لغة	<code>lang/en/*.php</code> ، <code>lang/ar/*.php</code>	التحقق، كلمات المرور، الترقيم، نصوص الشهادة، أسماء الدول
محتوى قاعدة البيانات	أعمدة JSON عبر <code>spatie/laravel-translatable</code>	المقالات، الأسئلة الشائعة، الفيديوهات...

1.2 الوسيط `ApiLocalization`

الملف: `app/Http/Middleware/API/ApiLocalization.php`

منطقه بالترتيب:

1. يقرأ ترويسة `Accept-Language`.
2. يأخذ أول مقطع قبل الفاصلة (فمثلاً `ar,en-US;q=0.9` تصبح `ar`).
3. يحذف جزء المنطقة: `en-US` تصبح `en`.
4. يحوّل النتيجة إلى أحرف صغيرة.
5. إن كانت الترويسة غائبة، أو كانت اللغة غير مدرجة في `config('app.supported_languages')`، يعود إلى `config('app.locale')`.
6. ينفّذ `app()->setLocale($locale)`.
7. أثر جانبي مهم: إن كان الطلب موثقاً (مستخدم مسجّل)، يكتب اللغة على صف المستخدم:

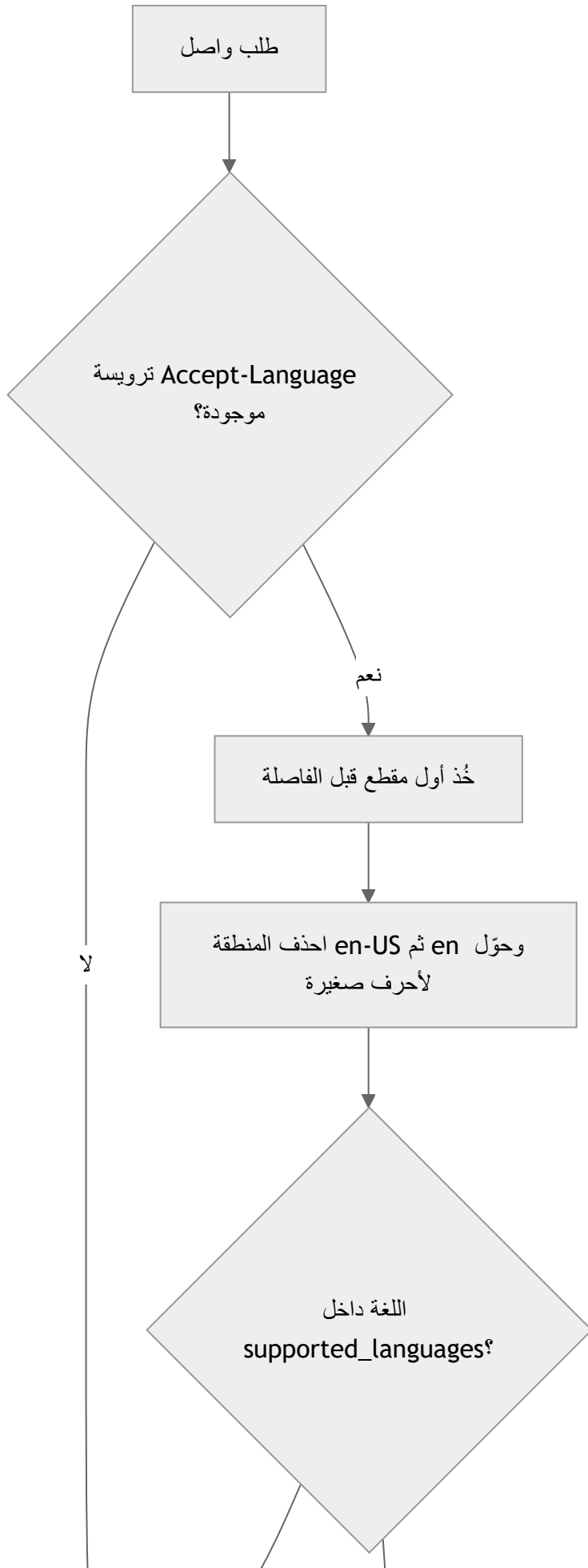
```
auth()->user()->update(['lang' => $locale]);
```

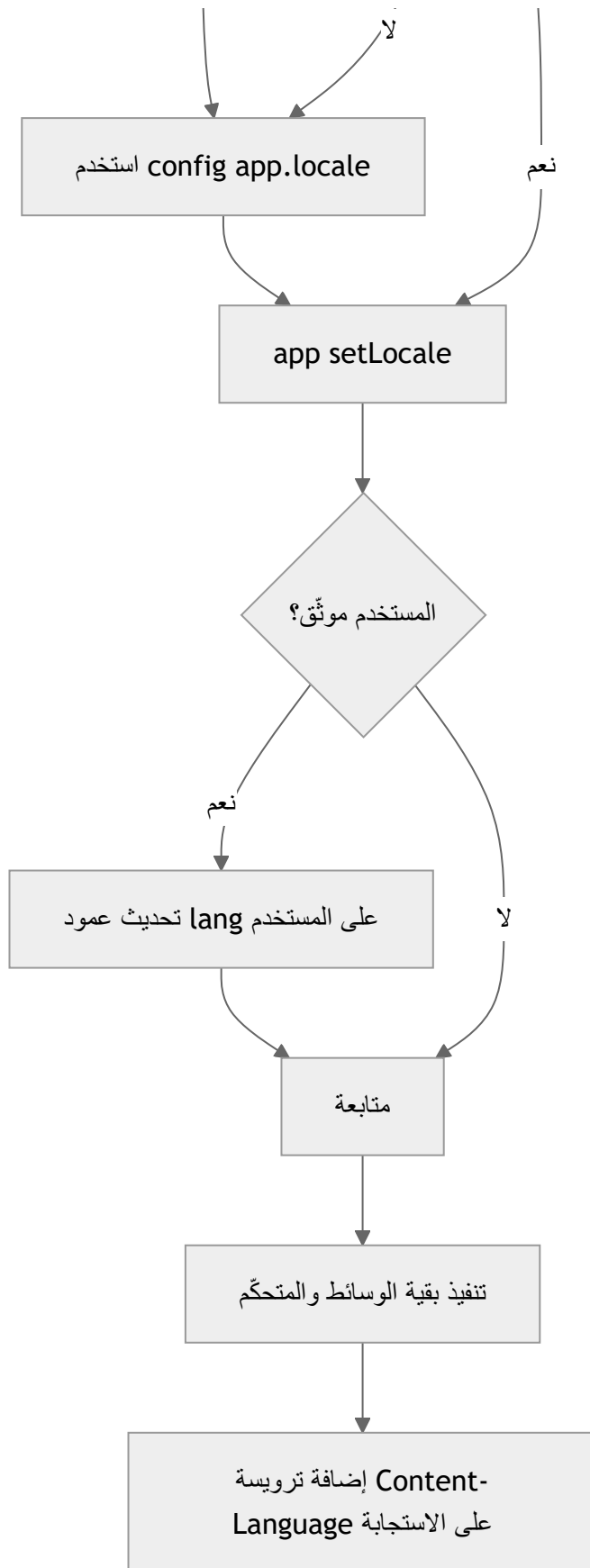
8. يضيف ترويسة `Content-Language` على الاستجابة.

هذا الوسيط مُسجّل في `bootstrap/app.php` وهو الوسيط الأول في كل مجموعات المسارات الأربع (`guest` ، `admin` ، `user` ، `web`).

⚠️ لاحظ الخطوة 7: هذه عملية كتابة على قاعدة البيانات في كل طلب موثّق، حتى لو لم تتغيّر اللغة. طالب مدقّق يجب أن يلاحظ أن هذا يزيد الحمل ويُعطّل بعض أنواع التخزين المؤقت، وأن الأنظف هو التحقق `if ($user->lang !== $locale)` قبل التحديث.

1.3 مخطط تدفق تحديد اللغة





1.4 الإعدادات في config/app.php

المفتاح	القيمة / متغير البيئة	ملاحظة
supported_languages	['en', 'ar'] (السطر 81)	مفتاح مخصص غير موجود في Laravel الافتراضي، يستهلكه ApiLocalization
locale	APP_LOCALE ، الافتراضي en	اللغة الاحتياطية عند فشل التفاوض
fallback_locale	الافتراضي en	
faker_locale	الافتراضي en	لبيانات الاختبار
timezone	APP_TIMEZONE ، الافتراضي UTC	
platform	PLATFORM ، الافتراضي http://localhost:3000/ (السطر 130)	يستخدم في الشهادات والروابط
platform_cert	PLATFORM_CERT (السطر 131)	خاص بصفحة الشهادة

1.5 قواميس الرسائل: اصطلاحان في المشروع نفسه

الاصطلاح الأول: ملفات JSON في جذر lang/، تُستدعى بلا مساحة أسماء:

```
return $this->sendResponse(true, $data, trans('Listed'));
```

أمثلة على المفاتيح: otpHasBeenSentToYourMobile , tryAgainAfter , disabledAccount , technicalError , successfullDelete , successfullUpload , successfullLogin , invalidOtp , otpHasBeenExpired , Rejected , UnderReview , youAreNotAllowedToDoThisAction , Listed , successfullUpdate .

⚠ مشكلة قائمة فعلاً: lang/en.json يحتوي 121 مفتاحًا بينما lang/ar.json يحتوي 109 مفاتيح — أي أن 12 مفتاحًا مفقود في العربية. سلوك Laravel عند غياب المفتاح هو إعادة المفتاح نفسه كنص، فيرى المستخدم العربي نصًا إنجليزيًا مثل technicalError بدلًا من رسالة مفهومة. مزامنة القاموسين مهمة صيانة حقيقية في هذا المشروع.

الاصطلاح الثاني: قواميس PHP لكل لغة داخل lang/ar/ و lang/en/ :

الغرض	الملف
رسائل المصادقة القياسية	auth.php
نصوص صفحة الشهادة وملف الـ PDF	certificate.php
أسماء الدول	map_countries.php
رسائل عامة	msg.php و msg.json
نصوص الترتيم	pagination.php
رسائل إعادة كلمة المرور	passwords.php
رسائل التحقق	validation.php

وُتستدعى بمساحة أسماء: `trans('msg.technicalError')`.

✦ نقطة تعليمية: وجود اصطلاحين (`trans('key')` و `trans('msg.key')`) لنفس النوع من الرسائل هو دين تقني. عند إضافة رسالة جديدة، ابحث أولاً أين توجد نظيراتها بدلاً من اختيار الاصطلاح عشوائيًا.

1.6 الأعمدة المترجمة عبر `spatie/laravel-translatable`

المفهوم: بدلاً من جدول منفصل للترجمات، تُخزّن قيم كل اللغات في عمود **JSON** واحد بالشكل `{"en": "...", "ar": "..."} .` وعند القراءة يعيد الموديل تلقائيًا القيمة الموافقة للـ locale الحالي — وهذا بالضبط سبب أهمية الوسيط `ApiLocalization`.

سنة موديلات تستخدم `HasTranslations`:

الموديل	الملف	الأعمدة المترجمة
Blog	app/Models/Blog.php	content , short_description , title
Faq	app/Models/Faq.php	description , title
Question	app/Models/Question.php	answers_c , answers_b , answers_a , question (وغيرها)
Scenes	app/Models/Scenes.php	title
Tawia	app/Models/Tawia.php	symptoms , description , title
Video	app/Models/Video.php	description , title , video_url

ملاحظتان دقيقتان:

- في `Tawia` أعلن `public $translatable` بينما تستخدم بقية الموديلات `protected` — عدم اتساق شكلي لا يغيّر السلوك لكنه يلفت النظر.
- في `Video` العمود `video_url` مترجم أيضًا، أي أن لكل لغة رابط فيديو مختلف (نسخة عربية ونسخة إنجليزية من المقطع). هذا قرار تصميمي مقصود ومهم لفهم منصة تدريبية.

1.7 الفجّ الشهير: لا يمكن الترتيب على الأعمدة المترجمة

المكان: `app/Http/Traits/Controller/IndexTrait.php` في الدالة `indexInit()` — وهي نقطة القوائم المشتركة لكل الموارد.

الترتيب مبني على قائمة بيضاء (whitelist):

```
$allowedSortColumns = array_unique(array_merge(['id'], $this->columns, array_keys($sortAllowed)))
// ثم يُتحقق منها:
'sort_column' => 'in:'.implode(',', $allowedSortColumns),
```

النتيجة العملية: إذا أرسل العميل عمودًا غير مدرج — مثل عمود JSON مترجم — فسيحصل على **422 Unprocessable Entity** من طبقة التحقق، وليس خطأ SQL. هذا سلوك جيد (فشل مُبكر ونظيف) لكنه يفاجئ مطوّر الواجهة الذي يتوقع أن كل عمود قابل للترتيب.

بقية سلوك `indexInit()` المهم:

السلوك	التفصيل
<code>\$sortAllowed</code>	قيمته إما <code>true</code> (ترتيب مباشر <code>orderBy</code> ، للأعمدة الافتراضية/المحسوبة) أو <code>callable(fn(\$query, \$direction))</code> لترتيب يحتاج <code>join</code>
التصفية	<code>likeStart</code> للأعمدة التي لا تنتهي بـ <code>_id</code>
البحث الحر <code>q</code>	<code>orLike</code> على كل أعمدة <code>\$this->columns</code> مجمعة بـ <code>OR</code>
المحذوفات	<code>withTrashed()</code> افتراضيًا
التصدير	عند <code>export=true</code> يرفع <code>memory_limit</code> إلى 2G ويبيّن عبر <code>\$this->resourceExport::collection(\$items->cursor())</code>
الترتيب الافتراضي	<code>primaryKey DESC</code>

وقيود القوائم من `config/constants.php`:

المفتاح	القيمة
<code>per_page</code>	افتراضي 20، الحد الأقصى 100 (عبر <code>list_validations</code>)
<code>page</code>	الحد الأقصى 1000
<code>date_to</code> / <code>date_from</code>	صيغة صارمة <code>Y-m-d\TH:i:s.v\Z</code>

2. رفع الملفات والتخزين

2.1 المفهوم

Laravel يجزّد التخزين عبر `Storage` و"الأقراص" (`disks`): تكتب الشيفرة إلى قرص منطقي، والإعداد يقرّر أين يقع فعليًا (نظام ملفات محلي، مجلد عام، أو S3). فائدة هذا التجريد أن الانتقال إلى سحابة لا يستلزم تغيير شيفرة الرفع.

2.2 الإعداد في `config/filesystems.php`

القرص	الجزر	الرابط العام
<code>local</code>	<code>storage/app</code>	لا يوجد (غير عام)
<code>public</code>	<code>storage/app/public</code>	<code>APP_URL . '/storage'</code> ، الظهور <code>public</code>
<code>s3</code>	خدمة أمازون	حسب الإعداد

القرص الافتراضي يأتي من `FILESYSTEM_DISK` وقيّمته `local` (وهي كذلك في `.env.example`).

تذكير عملي: القرص `public` لا يُخدّم للمتصفح إلا بعد تنفيذ `php artisan storage:link` الذي يُنشئ الوصلة الرمزية `public/storage → storage/app/public`.

2.3 أربع دوال مساعدة غير متسقة في `app/Helpers/general.php`

هذه من أهم الملاحظات في المشروع كلّ — أربع طرق مختلفة لرفع ملف:

الدالة	السطر	إلى أين تكتب	ملاحظات
<code>upload()</code>	196	<code>public_path('uploads/' . \$path)</code> <code>move()</code> عبر	تتجاوز Storage تمامًا
<code>upload64()</code>	213	نفس المكان	جسم الدالة مطابق حرفيًا لـ <code>upload()</code> — تكرار صريح
<code>uploadToStorage()</code>	230	القرص public	ينفذ <code>mkdir</code> ثم <code>chmod 0755</code> ، ويعيد <code>['relativePath', 'absolutePath' => asset('storage/...')]</code>
<code>uploadStorage()</code>	264	القرص local تحت <code>public/<path></code>	مسار مربك: مجلد باسم public داخل قرص غير عام

اسم الملف في كل الحالات يُبنى بالصيغة:

```
sha1(time() . $file->getClientOriginalName()) . '.' . $extension;
```

وهناك `FileBaseURL()` التي تُلحق `config('app.file_url')` كبادئة للروابط.

⚠ **عيب تصميمي صريح:** `uploadToStorage()` عند الفشل تُعيد كائن `response()->json(...)` من داخل دالة مساعدة بدلًا من رمي استثناء. هذا يخلط بين طبقة الأدوات وطبقة HTTP، ويعني أن المُستدعي قد يظن أنه استلم مسار ملف وهو في الحقيقة يحمل استجابة HTTP. الصحيح: رمي استثناء وترك تحويله إلى استجابة لطبقة معالجة الأخطاء.

كذلك في `app/Http/Controllers/BaseApiController.php` الدالة `export()` (السطر 93) تكتب ملفات التصدير وتبني رابطها في السطر 121 بالشكل `asset(Storage::url($path))`.

الدرس للطالب: عند إضافة ميزة رفع جديدة، لا تختار دالة عشوائيًا. تعرّف أولًا: هل الملف يجب أن يكون عامًا؟ إن نعم فاستخدم القرص **public** (`uploadToStorage`) والتزم به، ولا تُنشئ طريقًا خامسًا.

3. تحديد معدل الطلبات (Rate Limiting)

3.1 المفهوم

تحديد المعدل يمنع إسراف الاستخدام وهجمات التخمين (brute force) بحدّ عدد الطلبات المسموحة لكل عميل في نافذة زمنية. في Laravel يتحقّق ذلك بالوسيط `throttle`، ويُخصّص عبر `RateLimiter::for(...)` في أحد الـ `providers`.

3.2 الواقع في هذا المشروع

لا يوجد تحديد معدل على الإطلاق. لا وسيط `throttle` في أي من ملفات المسارات، ولا استدعاء واحد لـ `RateLimiter::for()` في الـ `providers`.

وهنا الالتباس الأكبر: الملف `app/Http/Middleware/RequestRateMiddleware.php` اسمه مُضِلّ. هو لا يحدّ شيئًا، بل يسجّل كل طلب في جدول `request_rates` ثم يُنادي `$next($request)` دائمًا. الأعمدة التي يسجّلها:

العمود	المحتوى
<code>method</code>	فعل HTTP
<code>url</code>	العنوان المطلوب
<code>rate</code>	القيمة الثابتة 1
<code>ip</code>	عنوان العميل
<code>referrer</code>	المُحيل
<code>body</code>	كامل جسم الطلب بصيغة JSON
<code>headers</code>	كل الترويسات

● مخاطر يجب أن يفهمها الطالب:

- خطر أمني/خصوصية: تسجيل الجسم والترويسات يعني أن كلمات المرور ورموز `Authorization: Bearer` تُخزّن نصًّا في قاعدة البيانات. أي وصول لقاعدة البيانات يصبح وصولًا للحسابات.
- خطر أداء: كتابة صف على القاعدة في كل طلب، بما فيها طلبات الزوّار، أي أن حركة عامة كبيرة تتحول إلى ضغط كتابة.
- خطر إدراكي: الاسم يجعل القارئ يفترض وجود حماية غير موجودة.

الإصلاح المنطقي: تسمية أو استبعاد الحقول الحساسة، وإضافة `throttle` حقيقي على المسارات الحساسة (تسجيل الدخول، إرسال OTP)، وإعادة تسمية الوسيط ليعبر عن وظيفته (تسجيل لا تحديد).

4. Telescope

4.1 المفهوم

`laravel/telescope` لوحة تشخيص تعرض الطلبات والاستعلامات والوظائف والاستثناءات والبريد... إلخ. هي أداة تطوير قوية، ولذلك كشفها في بيئة الإنتاج بلا حماية يعدّ ثغرة، لأنها تعرض بيانات الطلبات الفعلية.

4.2 الإعداد في `config/telescope.php`

المفتاح	القيمة
<code>enabled</code>	<code>TELESCOPE_ENABLED</code> ، الافتراضي <code>true</code>
<code>path</code>	<code>TELESCOPE_PATH</code> ، الافتراضي <code>te1513462</code> (مسار مُبهم بدل <code>telescope</code>)
<code>driver</code>	<code>database</code>
المراقبات (watchers)	كلها مُفعّلة افتراضيًا

إبهام المسار (`te1513462`) هو "أمان بالإخفاء" (security through obscurity): يقلّل الاكتشاف العابر لكنه ليس بديلًا عن المصادقة. ولذلك يوجد أيضًا في `bootstrap/app.php` مجموعة وسائط باسم `telescope-auth` تستعمل `AuthenticateWithBasicAuth` ، ومسارات Telescope نفسها تبقى محمية عبر `config('telescope.middleware')`.

4.3 الفخّ الموثّق: 403 على مسار الشهادة العام

في `bootstrap/app.php` تعليق صريح يحفظ درسًا مكلّفًا:

يجب عدم إلحاق `Telescope\Authorize` بمجموعة `web` كاملة.

السبب: عندما ألحق بها، أصبح المسار العام للشهادة `storage/certificates/{pdf}` يعيد **403 للزوّار**، لأن دالة `viewTelescope` تستقبل `$user` بلا نوع محدّد، وتمنع من ليس مستخدمًا موثّقًا. النتيجة: بوابة أداة تطوير خنقت ميزة عامة تخصّ المستخدم النهائي.

الدرس: الوسائط تُلحق بأضيّق نطاق ممكن. وسيط تخويل خاص بأداة واحدة يجب أن يبقّى على مسارات تلك الأداة.

ملاحظة إضافية: التحقق من `CSRF` مستبعد لـ `*` و `telescope/*` ، وهناك `$middleware->trustProxies(at: '*')` مع `statefulApi()`.

5. الاختبارات

5.1 كيف تُشغّل

```
cd backend
php artisan test
# أو
vendor/bin/phpunit
```

الإعداد في `phpunit.xml`.

5.2 ما هو موجود فعلاً

الملف	ما يغطّيه
<code>tests/Feature/AppTest.php</code>	مُشغّل عام <code>test_crud(\$resource, \$token, ...)</code> يضرب <code>index</code> ، <code>edit</code> ، <code>show</code> ، <code>create</code> ، <code>store</code> ، <code>update</code> ، <code>toggleActive</code> — ويرسل <code>Accept-Language: en</code> ، <code>destroy</code>
<code>tests/Feature/SortingTest.php</code>	الترتيب برمز إداري على <code>Partner</code> و <code>Story</code> و <code>Video</code> ، ويشمل <code>test_partners_invalid_sort_direction_returns_422</code> المستند إلى <code>constants.list_validations</code> . يستخدم <code>RefreshDatabase</code>
<code>tests/Feature/StoriesTest.php</code>	سلوك القصص
<code>tests/Feature/CertificateStorageTest.php</code>	يستخدم <code>Storage::fake('public')</code> ويتأكد أن <code>CertificateService::storePdf</code> يُنشئ مجلد <code>certificates/</code>
<code>tests/Unit/CertificateServiceTest.php</code>	منطق خدمة الشهادة بمعزل
<code>tests/Unit/ExampleTest.php</code>	مثال الافتراضي Laravel

✦ لماذا اختبار إنشاء مجلد `certificates/` مهم؟ لأن النشر الجديد ينقذ `storage:link` فقط ولا يُنشئ المجلدات الفرعية، و `file_put_contents` لا يُنشئ المجلدات الناقصة. فبدون إنشاء صريح للمجلد، توليد الشهادة يفشل على أول خادم جديد فقط — وهو أسوأ أنواع الأخطاء: يعمل عندك ويفشل في الإنتاج.

6. الأخطاء الشائعة والملاحظات المهمة للطالب

6.1 PLATFORM الفارغ: أخطر مصيدة في المشروع

الملف: `app/Services/CertificateService.php` — وهو يوثق الفخ بنفسه.

القاعدة التي يجب حفظها:

```
// env('PLATFORM', 'http://localhost:3000/')
// PLATFORM= هو env. إذا كان السطر في
// !النتيجة '' وليست القيمة الافتراضية
```

السبب: `env()` تعيد القيمة الافتراضية فقط عندما يكون المفتاح غائبًا تمامًا، لا عندما يكون موجودًا بقيمة فارغة.

الأثر الذي وقع فعليًا: منصة فارغة ⇒ روابط نسبية ⇒ `dompdf` لم يستطع جلب صورة الـ `canvas` ⇒ ملف PDF فارغ، وأكواد QR تشير إلى لا شيء.

العلاج المطبّق: الدالة `assertAbsoluteUrl()` (نحو السطر 58) ترمي `RuntimeException` تذكر اسم `PLATFORM` صراحةً إن لم تُطابق القيمة النمط `^https?://`. أي **الفشل الصريح المبكر** بدل الفشل الصامت.

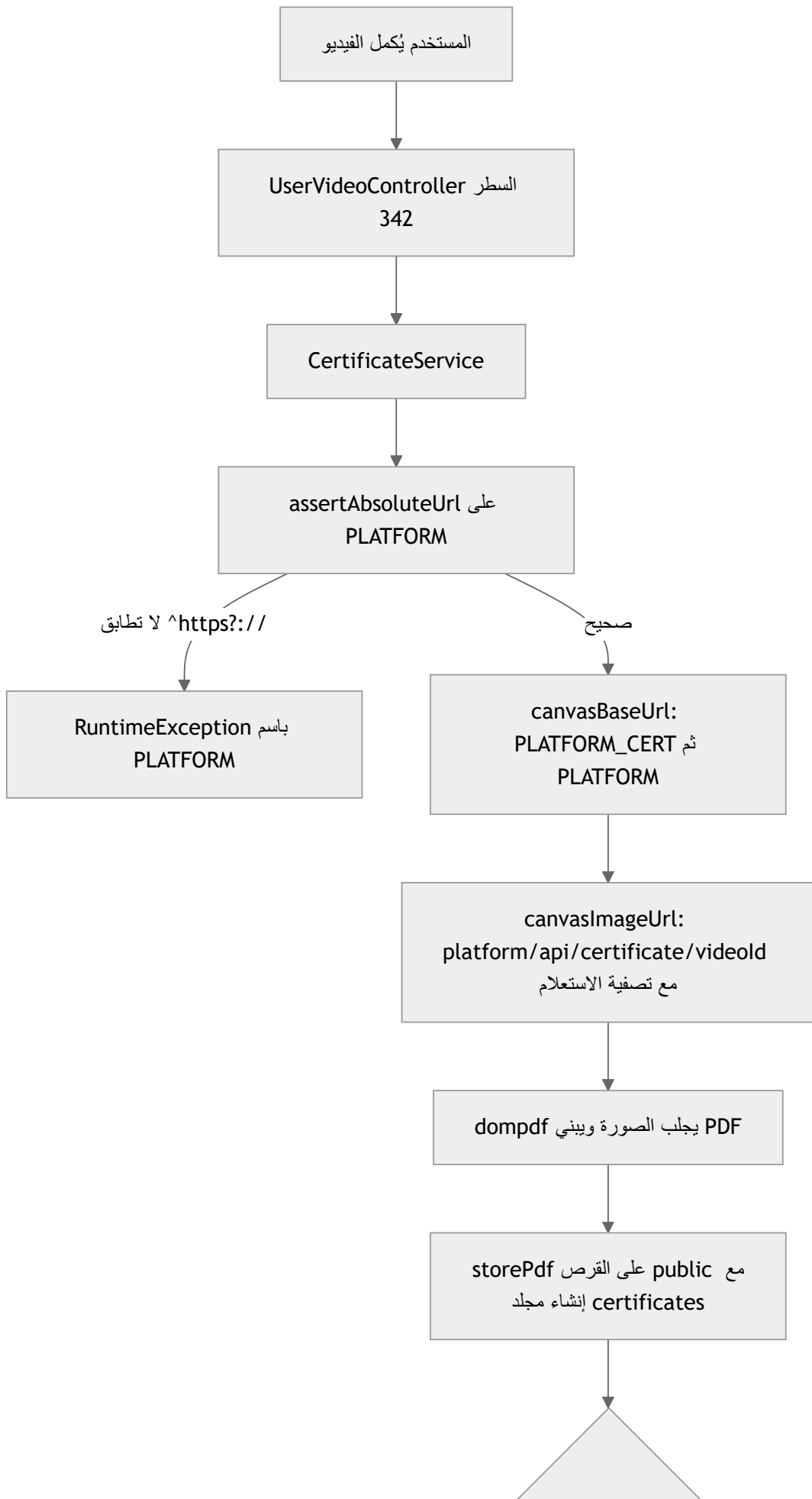
الدوال ذات الصلة:

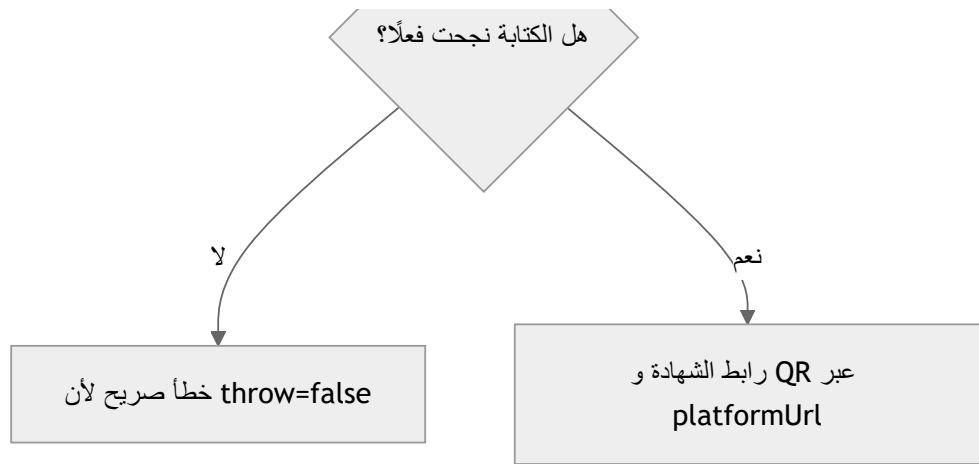
الدالة	الدور
<code>platformUrl()</code>	روابط QR والبريد
<code>canvasBaseUrl()</code>	تُفضّل <code>PLATFORM_CERT</code> ثم تعود إلى <code>PLATFORM</code> — لأن بعض المضيفين لا يستطيعون تحليل اسم مضيفهم العام
<code>canvasImageUrl()</code>	تبني <code><platform>/api/certificate/<videoId>?<query></code> مع تصفية قيم الاستعلام الفارغة/ <code>null</code>
<code>storePdf()</code>	تكتب عبر القرص <code>public</code> الذي أُعدّ بـ <code>throw => false</code> ، لذا تُتحقّق الكتابة ولا تُفترض

المستهلك: `app/Http/Controllers/User/UserVideoController.php:342`.

⚠️ المتغيّران `PLATFORM` و `PLATFORM_CERT` موجودان بقيمة فارغة في `.env.example` (السطران 74–75)
— أي أن النسخة الافتراضية للمشروع هي بالضبط حالة الفشل. أول ما تفعله بعد نسخ `.env.example` :
املاهما بروابط مطلقة.

6.2 مخطط تدقق توليد الشهادة





6.3 رموز حالة غير متوقعة

الوسيط	الملف	السلوك
AdminMiddleware	app/Http/Middleware/API/	يتحقق من حارس <code>auth('admin')</code> ، ويعيد HTTP 400 (لا 403) مع <code>"You are not admin user"</code>
UserMiddleware	app/Http/Middleware/API/	يتحقق من حارس <code>auth('user')</code> ، ويعيد HTTP 400 مع <code>"You are not user"</code>

هذا انحراف عن الدلالة المعيارية (Forbidden 403 للتحويل). لكنه سلوك قائم يعتمد عليه العميلان، فتغييره تغيير في العقد بين الخادم والواجهات ولا يُفعل منفردًا.

ومعالجات الاستثناءات في `bootstrap/app.php` تعيد المُغلّف القياسي:

الاستثناء	الرمز	الرسالة
AuthenticationException	401	Unauthorized
NotFoundHttpException	404	EndPoint Is Not Found

بشرط واحد مهم: فقط عندما يكون `expectsJson()` صحيحًا. أي أن طلبًا بلا ترويسة `Accept: application/json` قد يحصل على صفحة خطأ HTML بدل المُغلّف — سبب شائع لارتباك مطوّري الواجهة وأدوات مثل curl.

6.4 قائمة تحقّق سريعة للطالب

1. أرسل `Accept-Language` مع كل طلب؛ وإلا حصلت على `en` صامتًا.
2. أرسل `Accept: application/json`؛ وإلا فقدت مُغلّف الأخطاء.
3. لا تحاول الترتيب على عمود مترجم — النتيجة 422.
4. عند إضافة رسالة جديدة: أضفها إلى `en.json` و `ar.json`.

5. عند إضافة عمود مترجم جديد: أضيفه إلى `$translatable` ، واجعل نوعه JSON ، ولا تضعه في قائمة الترتيب.
6. بعد `composer install` : أنشئ `.env` ، ثم `key:generate` ، ثم `migrate` ، ثم `storage:link` ، ثم املأ `PLATFORM` .
7. لا تعتمد على وجود حماية معدل طلبات — لا وجود لها.
8. أوقف `TELESCOPE_ENABLED` في أي بيئة عامة، ولا تُلحق `Telescope\Authorize` بمجموعة `web` .

أسئلة للمراجعة

- (1) كيف يحدّد الخادم لغة الطلب، وماذا يحدث إذا أرسل العميل `Accept-Language: fr` ؟ `ApiLocalization` يأخذ أول مقطع ويحذف المنطقة ويصغّر الأحرف، ثم يتحقق من وجود اللغة في `config('app.supported_languages')` = `['en','ar']` . لأن `fr` غير مدرجة، يعود إلى `config('app.locale')` أي `en` — بصمت ودون خطأ.
- (2) ما الأثر الجانبي غير المتوقع في وسيط التعريب؟ إن كان الطلب موثّقًا فهو ينفذ `auth()->user()->update(['lang' => $locale])` في كل طلب، أي كتابة على قاعدة البيانات لكل نداء API موثّق حتى بلا تغيير فعلي للغة.
- (3) لماذا يُرجع طلب الترتيب على عمود `title` المترجم رمز 422 وليس خطأ SQL؟ لأن `IndexTrait::indexInit()` يبني قائمة بيضاء `$allowedSortColumns` من `['id']` و `$this->columns` و مفاتيح `$sortAllowed` ، ثم يتحقق `'sort_column' => 'in:…'` . الرفض يحدث في طبقة التحقق قبل الوصول إلى قاعدة البيانات.
- (4) ما الفرق بين `trans('technicalError')` و `trans('msg.technicalError')` ؟ الأولى تقرأ من ملف JSON في جذر `lang/` (`en.json` / `ar.json`) ، والثانية من قاموس PHP `lang/<locale>/msg.php` . المشروع يستخدم الاصطلاحين معًا وهذا دين تقني يجب مراعاته عند إضافة رسائل.
- (5) ما المشكلة الفعلية بين `lang/en.json` و `lang/ar.json` ؟ غير متزامنين: 121 مفتاحًا بالإنجليزية مقابل 109 بالعربية، أي 12 مفتاحًا مفقودًا. عند غياب المفتاح يعيد Laravel المفتاح نفسه، فيظهر نص إنجليزي خام للمستخدم العربي.
- (6) هل يوجد تحديد معدل طلبات في المشروع؟ وما دور `RequestRateMiddleware` ؟ لا يوجد إطلاقًا: لا `throttle:` ولا `RateLimiter::for()` . `RequestRateMiddleware` مُضِلّ الاسم — هو يسجّل كل طلب (الفعل، الرابط، IP، المُحيل، كامل الجسم، وكل الترويسات) في جدول `request_rates` ثم يمرّر الطلب دائمًا. مخاطره: تسريب رموز `Bearer` وكلمات المرور إلى قاعدة البيانات، وكتابة إضافية على كل طلب.

(7) لماذا مُنِع إلحاق Telescope\Authorize بمجموعة web ؟ لأن ذلك جعل المسار العام storage/certificates/{pdf} يعيد 403 للزوّار: دالة viewTelescope تستقبل \$user بلا نوع وتمنع غير الموثّقين. فمُنِع الإلحاق العام، وبقيت مسارات Telescope محميّة عبر config('telescope.middleware') ومجموعة telescope-auth بـ AuthenticateWithBasicAuth .

(8) اشرح فحّ PLATFORM= الفارغ وكيف عُولج. env() تعيد قيمتها الافتراضية فقط إذا كان المفتاح غائبًا؛ أما المفتاح الموجود بقيمة فارغة فيُنتج '' . النتيجة كانت روابط نسبية، فعجز dompdf عن جلب صورة ال canvas (PDF فارغ) وأشارت أكواد QR إلى لا شيء. العلاج: assertAbsoluteUrl() ترمي RuntimeException تذكر PLATFORM إن لم تُطابق القيمة ^https?:// .

(9) لماذا يوجد PLATFORM_CERT إلى جانب PLATFORM ؟ لأن canvasBaseUrl() تفضّل PLATFORM_CERT ثم تعود إلى PLATFORM ؛ وذلك لبيئات لا يستطيع فيها الخادم تحليل اسم مضيفه العام من الداخل، فيحتاج عنوانًا داخليًا لجلب صورة الشهادة.

(10) لماذا يهمّ اختبار CertificateStorageTest أن storePdf يُنشئ مجلد certificates/ ؟ لأن النشر الجديد ينقذ storage:link فقط ولا ينشئ المجلدات الفرعية، و file_put_contents لا تُنشئ المجلدات الناقصة. بدون إنشاء صريح، يعمل الكود محليًا ويفشل على خادم جديد — لذا يثبت الاختبار هذا الشرط باستخدام Storage::fake('public') .